



Project Report

BCT – Final Project Report

FACE-RECOGNITION ATTENDANCE KIOSK

Anirudha Sarkar

Aditya Talukdar

Sharanya Saha

Rohan Bose

Sukla Bala

Sukanya Das

Bipasa Paul

A report submitted in part fulfilment of the course of

Beyond Curriculum Training in Robotics

By

Students of B.Tech 3rd Year

Department of Electronics and Communication Engineering

Narula Institute of Technology

Declaration

This report has been prepared based on my own work. Wherever published or unpublished materials from other sources have been used, they have been properly acknowledged and cited.

I hereby declare that this project report entitled “Face-Recognition Attendance Kiosk” is submitted in partial fulfilment of the requirements for the Beyond Curriculum Training (BCT) in Robotics under the Department of Electronics and Communication Engineering, Narula Institute of Technology. This work has not been submitted to any other institution for the award of any degree or certification.

Student Name: Anirudha Sarkar

Date of Submission: 17-07-2026

Signature: Anirudha Sarkar

Acknowledgement

We are grateful to the Department of Electronics and Communication Engineering, Narula Institute of Technology, for providing the platform and resources for the Beyond Curriculum Training (BCT) in Robotics programme, under which this project was carried out.

We sincerely thank our faculty guide and mentors for their consistent support, technical guidance and constructive feedback at every stage of this project, from the initial concept through to testing on the actual hardware.

We would also like to thank the students who volunteered their photographs for the face database and helped test the kiosk in a real classroom setting — their cooperation was essential in validating the system under realistic conditions.

Finally, we thank our institution for creating an environment that encourages hands-on, practical learning of the kind this project represents.

Table of Contents

Acknowledgement	4
Abstract	8
List of Figures	6
List of Tables	7
Chapter 1: Introduction	9
Chapter 2: Literature Review	10
Chapter 3: Objectives	12
Chapter 4: System Analysis	13
Chapter 5: Proposed System	15
Chapter 6: Work Flow / Methodology	17
Chapter 7: Design and Implementation	18
Chapter 8: Testing	21
Chapter 9: Simulation and Results	23
Chapter 10: Conclusion and Future Scope	25
Bibliography	26
Appendix A: Configuration Parameters	27

List of Figures

Figure 1: System block diagram	15
Figure 2: Kiosk state machine	16
Figure 3: Face recognition pipeline	19

List of Tables

Table 1: Hardware and software requirements	13
Table 2: Software module responsibilities	18
Table 3: Test cases and results	21
Table 4: Sample attendance sheet	23
Table 5: Performance summary	23
Table 6: Configuration parameters (config.py)	27

Abstract

Attendance in classrooms and at events is still commonly taken by hand or with contact-based devices, which are slow, error-prone and easy to misuse through proxy marking. This project presents a Face-Recognition Attendance Kiosk — a self-contained station built on a Raspberry Pi 5 that identifies a person from their face and records their attendance automatically, without cards, apps or an internet connection.

The kiosk continuously watches for a face using a camera. When a person approaches, an animated pair of on-screen eyes tracks them, the system recognizes the face against a stored database, shows a personalized greeting with the person's photo and details, and writes a time-stamped entry to a daily attendance sheet. Each person is logged only once per day. The complete pipeline — detection, recognition and logging — runs entirely on the device, keeping the data private and the system usable even where there is no network.

Developed as part of Beyond Curriculum Training (BCT) in Robotics, the project combines computer vision, embedded systems and human-computer interaction in a single practical product. Its scope covers face-based identification, an interactive kiosk interface, on-device enrollment of new members and automatic record-keeping suitable for a classroom, laboratory or event entrance.

Chapter 1: **Introduction**

Attendance recording is one of the most repeated administrative tasks in any classroom, laboratory or training programme, yet it is still frequently done by hand — a register passed around, or a card tapped at a reader. Manual registers are slow when a class is large, easy to falsify through proxy marking, and produce records that must later be transcribed before they are useful for reporting. Token-based systems such as RFID cards remove some of the manual effort but introduce a new failure mode: the token itself. A card can be lost, forgotten, borrowed or shared, none of which say anything about who was actually present.

This project, undertaken as part of Beyond Curriculum Training (BCT) in Robotics, addresses the problem by removing the token altogether. The Face-Recognition Attendance Kiosk identifies a person directly from their face, using a camera and a small offline computer, and records their attendance automatically the moment they are recognized. No card, phone or network connection is required, which keeps the system usable in a classroom or lab that may not have reliable internet access, and keeps all biometric data on the device rather than on a cloud service.

Building the kiosk required combining several fields that a Beyond Curriculum Training programme is meant to expose students to: computer vision (detecting and recognizing a face in a live video stream), embedded systems (running the whole pipeline in real time on a Raspberry Pi 5 rather than a desktop or a cloud GPU), and interface design (presenting the result to the person standing at the kiosk in a way that is immediate and easy to understand). The result is a small, self-contained station that a classroom or event entrance can place at its door.

This report documents the complete development of the kiosk: the reasoning behind the design (Chapters 2 to 5), how it was built and what it is made of (Chapters 6 and 7), how it was tested (Chapter 8), the results obtained (Chapter 9), and what could be added in a future iteration (Chapter 10).

Chapter 2: Literature Review

Automated attendance systems generally fall into three broad categories, each with a different trade-off between convenience, cost and reliability.

Manual and paper-based systems remain the most common in smaller institutions because they require no hardware at all, but they scale poorly: marking a class of sixty students by roll call takes several minutes of lesson time, and a paper register offers no protection against a student answering for an absent classmate.

Token-based systems — RFID cards, barcode ID cards and, on some campuses, NFC-enabled phones — automate the recording step and are cheap to deploy per reader. Their weakness is that they authenticate the token, not the person; a card handed to a friend marks the friend's attendance under the card owner's name. Fingerprint scanners solve the token-sharing problem by checking a biometric trait directly, and are widely used in offices and some colleges, but they require physical contact with a shared sensor surface, which raises hygiene concerns and slows down a queue of people entering at the same time.

Face recognition has become practical for this kind of application only in the last decade, as two things improved together: cameras became cheap enough to put on every device, and the algorithms used to turn a face into a comparable numeric representation became far more accurate. Early systems used comparatively simple statistical techniques such as Eigenfaces and Fisherfaces, which are sensitive to lighting and pose. Modern systems, including the one built for this project, instead use a deep convolutional network trained to output a fixed-length embedding — a vector of numbers — such that two photographs of the same person produce nearby vectors and two different people produce vectors that are far apart. This project uses the open-source “face_recognition” library, which wraps a ResNet-based model from dlib trained for exactly this purpose, avoiding the need to train a recognition model from scratch.

Most commercial face-recognition attendance products, such as cloud-connected office kiosks, send the camera feed or the extracted face data to a remote server for matching, which adds a recurring cost, a dependency on network connectivity, and a privacy question about where biometric data is stored. This project takes the opposite approach: detection, encoding,

matching and logging all happen on the Raspberry Pi itself, so the system keeps working with no internet connection and no student's face data ever leaves the device it was enrolled on.

Chapter 3: **Objectives**

- Main objective: to design a low-cost, offline kiosk that marks attendance automatically by recognizing a person's face in real time.
- Specific goals: detect and track a face from a live camera; recognize known individuals and reject unknown faces; greet each person on screen; log attendance once per day with no duplicates; allow new members to be enrolled at the kiosk; and run entirely on a Raspberry Pi without internet.
- Expected output: within a few seconds of a person stepping up, the kiosk identifies them, greets them by name and saves a time-stamped record — or clearly shows “not recognized” when there is no match.

Chapter 4: System Analysis

Existing attendance methods — manual registers, RFID cards and fingerprint scanners — are slow, easy to misuse through proxy marking, and, in the case of fingerprints, require physical contact. A face-based system removes contact, is hard to fake casually, and needs no token that can be shared or lost.

Feasibility: the system is technically feasible using an off-the-shelf Raspberry Pi 5, a camera and mature open-source vision libraries; economically feasible because the hardware is inexpensive and the software is free; and operationally simple, needing no training to use.

Functional requirements: detect a face, recognize it against a stored database, display a personalized greeting, record attendance once per person per day, enroll new members at the kiosk, and restrict recognition to selected groups.

Non-functional requirements: operate fully offline, keep the interface smooth and responsive, protect privacy by storing all data on the device, and resume correctly after a restart.

Table 1: Hardware and software requirements.

Category	Item	Notes
Hardware	Raspberry Pi 5 (8 GB)	Main compute board running the whole pipeline
Hardware	Camera module / USB webcam	Live video input for detection and recognition
Hardware	HDMI display or touchscreen	Shows the animated eyes and profile screens
Hardware	microSD card, 32 GB+	Boots Raspberry Pi OS and stores the project files
Hardware	5V/5A USB-C power supply	Official Pi 5 supply recommended for stable operation
Software	Raspberry Pi OS (Bookworm, 64-bit)	Base operating system
Software	Python 3.11	Application runtime
Software	OpenCV	Camera capture and image

		handling
Software	MediaPipe	Real-time face-mesh tracking for the animated eyes
Software	face_recognition (dlib)	Face encoding and nearest-neighbour matching
Software	pygame	Kiosk display, animation and input handling

Chapter 5: Proposed System

- Block diagram: the overall data flow is shown in Figure 1 below — from the camera through detection and recognition to the attendance decision and the on-screen display.
- Description: the kiosk runs a continuous loop with four states — Idle (animated eyes wait for a face), Verify (the face is recognized), Greet (a known person's photo, name and details are shown and attendance is marked) and Not-Recognized (shown when no match is found within five seconds). A short cool-down then returns it to Idle.
- Architecture: a dedicated thread continuously captures camera frames, a background worker performs recognition so the interface never freezes, and the main loop drives the on-screen eyes and screens. The logic is split across five Python modules for vision, interface, attendance logging, configuration and overall control.

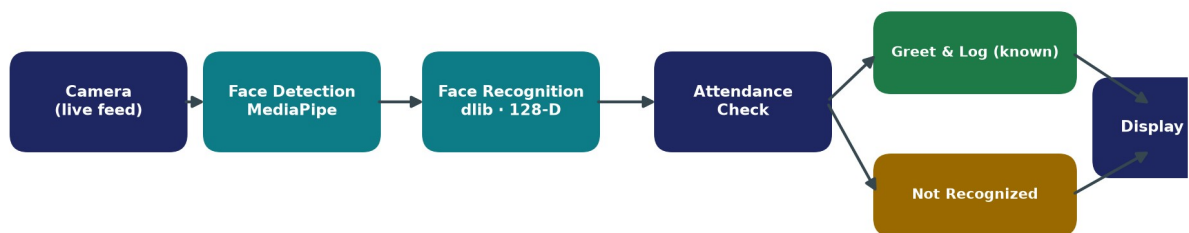


Figure 1: System block diagram.

- State diagram: Figure 2 below shows the four kiosk states and the transitions between them, matching the Idle / Verify / Greet / Not-Recognized description above.

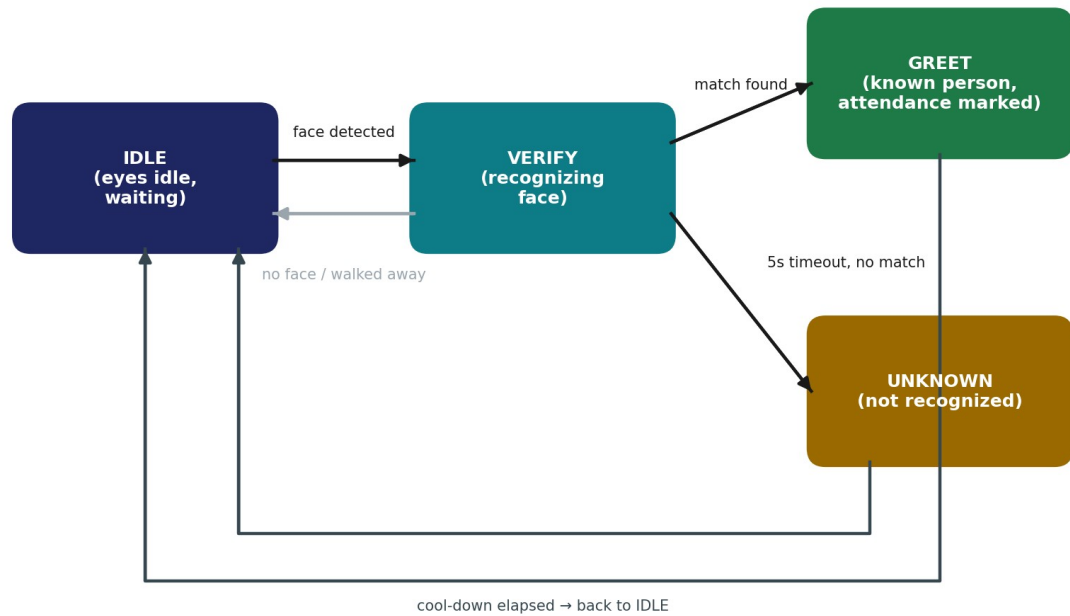


Figure 2: Kiosk state machine.

Chapter 6: **Work Flow / Methodology**

- Step-by-step process: collect reference photos of each member → build the face database → implement face detection → add recognition and matching → design the on-screen interface → add attendance logging → add on-kiosk enrollment → test on the Raspberry Pi.
- Development flow: Requirement analysis → Design → Implementation → Testing → Deployment on the kiosk.
- Tools used: Python 3.11, OpenCV, MediaPipe, the face_recognition library (dlib), pygame, and Raspberry Pi OS.

Chapter 7: Design and Implementation

- Software modules: `main.py` runs the state machine; `bot_vision.py` handles the camera, face detection, recognition and enrollment; `bot_eyes.py` draws the animated eyes and screens; `bot_attendance.py` writes the daily log; and `config.py` holds the settings.
- Recognition pipeline: MediaPipe locates and frames the face, the `face_recognition` library converts it to a 128-number feature vector, and the vector is matched to the closest known person — a distance below 0.42 counts as a match.
- Implementation highlights: threaded capture and background recognition keep the interface smooth; attendance is written to a daily CSV file with no duplicate entries; new members can be enrolled at the kiosk (E key) and the database reloaded without a restart (R key).

Table 2: Software module responsibilities.

Module (file)	Responsibility
<code>main.py</code>	Runs the state machine (IDLE → VERIFY → GREET/UNKNOWN → cool-down) and coordinates the other modules.
<code>bot_vision.py</code>	Owns the camera, MediaPipe face tracking, face encoding/matching, on-kiosk enrollment and the face database.
<code>bot_eyes.py</code>	Draws the animated eyes, the profile/greeting screen, the idle statistics and the enrollment overlay using pygame.
<code>bot_attendance.py</code>	Appends to the daily attendance CSV and prevents duplicate entries for the same person on the same day.
<code>config.py</code>	Holds every tunable constant — thresholds, timings, folder paths, enabled groups — in one place.

Recognition pipeline in detail (Figure 3): a face is not compared to the database as raw pixels. Each frame that reaches the recognition worker is first handed to MediaPipe's FaceMesh model, which locates 468 landmark points

on the face and is used for two things — pointing the animated eyes at the person, and confirming that a face is actually present before the slower recognition step runs. Once a face region is available, the face_recognition library — built on a dlib ResNet model — converts it into a 128-number vector that describes the face numerically. This vector is compared against every enrolled person's stored vector using Euclidean distance, and the closest match is accepted only if its distance is below 0.42; anything farther than that is treated as an unrecognized face rather than forced into the nearest available match. This threshold was tuned empirically during testing (Chapter 8) to reject look-alikes without being so strict that normal lighting changes caused a known person to be missed.

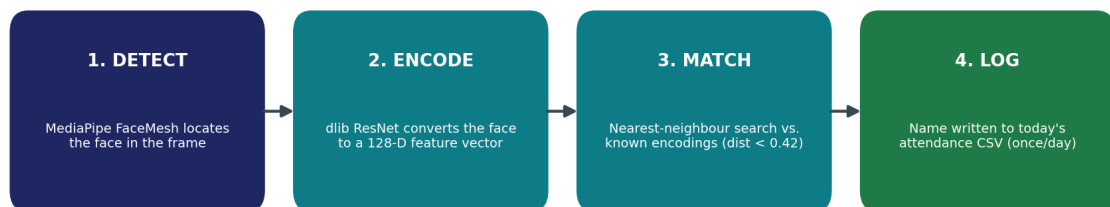


Figure 3: Face recognition pipeline.

The matching step itself is a short nearest-neighbour search (abridged from `bot_vision.py`):

```
face_distances = face_recognition.face_distance(known_encodings, encs[0])
best_match_index = np.argmin(face_distances)
score = face_distances[best_match_index]
if score < FACE_MATCH_THRESHOLD:      # 0.42
    return known_names[best_match_index]
return "Unknown"
```

Threading model: a kiosk that freezes while it thinks is a bad kiosk, so the application never runs recognition on the same thread that draws the screen. Three threads run concurrently: a camera-grabber thread continuously reads frames from the camera and keeps only the latest one; a recognition-worker thread picks up a frame roughly four times a second (every 0.25 s) and runs the detect–encode–match pipeline on it; and the main thread drives the pygame event loop and redraws the animated eyes at 30 frames per second regardless of whether a recognition attempt is in progress. All three threads share the camera frame and the recognition result through small, lock-protected handoffs, and every call into the dlib-based face_recognition library is wrapped in a single lock, since dlib's C++ internals are not safe to call from two threads at once.

The relevant transition inside the state machine looks like this (abridged from `main.py`):

```
elif state == STATE_VERIFY:
    if has_face and now - last_id_attempt >= ID_INTERVAL:
        vision.request_identify(frame)          # non-blocking
        name = vision.get_identify_result()
        if name and name != "Unknown":
            marked_now, time_str = attendance.mark(name, category)
            greet(name, frame, ..., marked_now)
    elif now - state_since > VERIFY_TIMEOUT:
        state = STATE_UNKNOWN
```

Enrollment and hot-reload: new members do not require restarting the kiosk or editing files by hand. Pressing E on the kiosk opens an on-screen name entry; on confirmation, the current camera frame is cropped around the detected face, saved as a photo under `known_faces/Students/<name>/`, and its encoding is registered in memory immediately, so the same person can be recognized on their very next attempt. Pressing R re-scans the entire `known_faces` folder on the background worker thread and rebuilds the encoding list without blocking the interface, which is useful after copying in a batch of new photos rather than enrolling one at a time at the kiosk itself.

Chapter 8: Testing

Testing combined small, targeted checks on the recognition pipeline with end-to-end runs of the complete kiosk on the Raspberry Pi. Table 3 lists the main scenarios exercised and their outcome.

Table 3: Test cases and results.

ID	Scenario	Expected result	Status
TC-01	Enrolled student faces the camera	Recognized within a few seconds; attendance marked and greeting shown	Pass
TC-02	Same student appears again later the same day	“Already marked today” shown; no duplicate CSV row	Pass
TC-03	Unenrolled / unknown face	“Face not recognized” shown after the 5 s verify timeout	Pass
TC-04	New student enrolled at the kiosk (E key)	Saved to known_faces/Students; recognized on next attempt, no restart	Pass
TC-05	Face database reloaded after adding photos to a folder (R key)	New identities appear in the loaded count	Pass*
TC-06	Enrollment / reference photo taken directly on a phone	Face detected despite EXIF rotation	Pass*
TC-07	Kiosk running across a midnight rollover	New attendance CSV created for the new date; previous file untouched	Pass
TC-08	Camera view briefly blocked, then cleared	Kiosk returns to IDLE and resumes tracking without crashing	Pass

* Debugging case study — EXIF orientation: during testing, three newly added student photographs (taken directly on a phone) failed to register in the face database even though the photos clearly contained a face. Investigation

traced the cause to the phone camera saving the image sensor data in landscape orientation and relying on an EXIF “orientation” tag to rotate it for display; the loading code read the raw pixel data without applying that rotation, so the face detector was handed a sideways image and found nothing. The fix applies PIL's `exif_transpose()` to every photo before detection, which resolved all three cases. The same pass also downsampled full-resolution phone photos (up to 4032×3024 pixels) to a maximum of 800 pixels on the long side before running detection, cutting the time to rebuild a ten-person database from about 71 seconds to under 9 seconds with no change in accuracy. This is a good example of why testing with real, unedited phone photographs — rather than only the pre-cropped reference images used early in development — is necessary before trusting a system to work in practice.

Chapter 9: Simulation and Results

The kiosk was tested on a Raspberry Pi 5 with a set of enrolled students. Each enrolled person was correctly recognized and marked present, a second appearance on the same day showed “already marked” instead of creating a duplicate, and unfamiliar faces were correctly reported as “not recognized”.

Performance: the interface ran at about 30 frames per second with recognition attempted several times a second on a background thread, so the animated eyes stayed smooth throughout. Recognition worked reliably in normal indoor lighting, and the 0.42 matching threshold gave a good balance between accepting the right person and rejecting look-alikes.

A sample of a day’s attendance sheet is shown below.

Name	Category	Time
Anirudha Sarkar	Signass Student	07:30 AM
Anil Kumar	Signass Student	09:12 AM
Abhinav Anand	Students	09:31 AM
Aditya Talukdar	Students	07:42 AM
Sharanya Saha	Students	07:55 AM
Rohan Bose	Students	08:10 AM
Sukla Bala	Students	08:14 AM
Sukanya Das	Students	08:20 AM
Bipasa Paul	Students	08:27 AM

Table 4: Sample attendance sheet (attendance/2026-07-17.csv).

Table 5: Performance summary.

Metric	Value
UI frame rate	~30 FPS, steady, independent of recognition load

Recognition attempts	4 per second (ID_INTERVAL = 0.25 s)
Face match threshold	0.42 (Euclidean distance on a 128-D encoding)
Face database reload (10 identities)	~9 seconds, after image downscaling
Attendance deduplication	1 record per person per day
Re-greet cool-down	60 seconds before the same person's screen reappears
Concurrent threads	3 — camera grabber, recognition worker, UI/render loop

Chapter 10: **Conclusion and Future Scope**

This project delivered a working Face-Recognition Attendance Kiosk that marks attendance automatically and contactlessly, entirely on a low-cost Raspberry Pi and without any internet connection. It removes the delays and proxy-marking of manual registers, keeps all data on the device for privacy, and presents a friendly, interactive interface.

Limitations: recognition needs reasonable lighting, the kiosk handles one face at a time, and because it works from a 2-D image it could in principle be fooled by a photograph.

Future scope: adding a liveness check to resist photo spoofing, handling several faces at once for busy entrances, exporting attendance to a spreadsheet or cloud service, and a small web dashboard for live monitoring.

Bibliography

- [1] MediaPipe — Google. <https://developers.google.com/mediapipe>
- [2] A. Geitgey, `face_recognition` (dlib-based). https://github.com/ageitgey/face_recognition
- [3] OpenCV — Open Source Computer Vision Library. <https://opencv.org>
- [4] pygame. <https://www.pygame.org>
- [5] Raspberry Pi Foundation. <https://www.raspberrypi.com>
- [6] D. E. King, “Dlib-ml: A Machine Learning Toolkit,” *Journal of Machine Learning Research*, vol. 10, 2009. <http://dlib.net>
- [7] F. Schroff, D. Kalenichenko and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” *IEEE CVPR*, 2015.

Appendix A: Configuration Parameters

Every tunable value used by the kiosk is defined in one place, `config.py`, rather than scattered through the code. Table 6 lists each parameter, its value at the time of testing, and what it controls.

Table 6: Configuration parameters (config.py).

Parameter	Value	Controls
CAMERA_INDEX	0	Which camera device OpenCV opens
FACE_MATCH_THRESHOLD	0.42	Maximum encoding distance accepted as a match (lower = stricter)
ID_INTERVAL	0.25 s	Time between recognition attempts while verifying
VERIFY_TIMEOUT	5.0 s	Time before an unmatched face is reported as not recognized
GREET_DURATION	6.0 s	How long the greeting/profile screen stays on screen
UNKNOWN_DURATION	3.0 s	How long the “not recognized” message stays on screen
RETRY_COOLDOWN	4.0 s	Pause before the kiosk starts verifying again after a result
REGREET_COOLDOWN	60.0 s	Minimum time before the same person's greeting screen can reappear
ENABLED_GROUPS	Signass Student, Students	Top-level known_faces folders that are loaded
ENROLL_GROUP	Students	Folder new on-kiosk enrollments are saved to